DBA5103

Operations Research and Analytics

HDB EV Charging

Placement and Cost-Effectiveness

A capacitated facility-location study

2027 planning case and 2030 outlook

Semester I, Academic Year 2026/2027

MSBA | National University of Singapore

11 November 2026

Reproducible model results for team review. Source vintages, assumptions and numerical certificates are preserved in the companion workbook and Python appendix.

1. Problem and decision

This study asks where HDB carparks need additional slow charging and how much service each investment level delivers. National charger counts cannot establish local access. Infrastructure expenditure covers participating parties, not a proposed government budget.

All 2,273 HDB carparks are retained. Existing address-supported AC/DC supply is fixed; new base additions are 7.4 kW AC points. Integer investment decisions and continuous energy allocation link nearby carparks. Non-HDB investment, carbon valuation and petrol savings are excluded.

Base result: 2,332 additional AC points at S$15.158m serve 90.0% of estimated 2027 demand, versus 61.1% using matched existing supply alone.

The allocation is designed for end-2027 demand. The September 2026 benchmark has a modelled energy shortfall of 17.0%; unchanged supply leaves 38.9% unmet in 2027. The proposed additions reduce the 2027 gap to 10.0%. These are estimated energy gaps, not observed charger shortages. Planning-area allocations are the main decision output; individual sites remain implementation details requiring verification.

2. Data and demand estimation

Input	Frozen evidence / treatment
HDB candidate sites [1]	2,273 unique IDs; projected SVY21 coordinates.
Car-lot snapshot [1]	31 Oct 2026; 706,663 matched lots; 276 missing and 2 zero records.

Input	Frozen evidence / treatment
Public charger register [2]	Sep 2026; 11,489 unique points, 9,279 registrations; 5,342 base matched points.
Vehicle stock / flows [3]	Cars/Electric only; Sep 2026 stock 74,887. Other fuels retained only in the all-car denominator.
Housing / population [4]	2025 with MP2019 geography; 2026 with MP2025 separately.
Costs and operation [5–7]	Vendor components and historical public-carpark usage inform scenarios; no site-level HDB quote or utilisation panel.

Base supply requires a unique literal block and normalised street match within 150 metres, with non-HDB-looking buildings excluded. Registration conflicts are not assigned. The broader sensitivity adds block-range and same-street candidates. A 48-location desk sample checks the source records; neither this check nor proximity establishes ownership on site.

Missing or zero lot counts use an area/carpark-type median with at least five observations, falling back to type and then national medians. This adds 71,123.5 assumed lots. All demand remains represented; no-build at unknown-capacity sites and 0.5×/1.5× imputations test the effect. The complete exceptions and match ledgers accompany the workbook.

2.1 Demand, capacity and scenario assumptions

$$E_{2027} = 74{,}887 + 15\,(2{,}872.78)(1 - 0.02) \approx 117{,}116.83$$

$$E_{2030} \in \{0.25, 0.35, 0.45\} \times 651{,}003$$

Stock scenarios: 15 months of average registrations with assumed 2% exits; 2030 uses 25/35/45% of the frozen car fleet. Displayed inputs are rounded.

$$E_t^{HDB} = h_t E_t, \quad EV_{it} = w_{it} E_t^{HDB}, \quad \sum_i w_{it} = 1$$

$$D_{it} = EV_{it}\, \beta\, \frac{m}{365}\, \frac{e}{\eta}$$

Demand D is grid-side kWh/day. Spatial weights sum to one; lots are not vehicle counts.

Parameter	Base	Interpretation
National EV stock E	117,116.83	2027 assumption from stock-flow identity
HDB share h / dependence β	0.40 / 0.80	HDB-associated EVs / energy sought here
Mileage m / intensity e	16,200 / 0.16	km/year / battery kWh per km
Efficiency η	0.90	Battery energy divided by grid energy
Catchment / lot share	400 m / 10%	Straight-line access / assumed space limit
Spare power / upgrade	44.4 / 88.8 kW	Incremental power beyond existing load

The base allocates 46,847 estimated HDB-associated EVs by normalised car-lot weights, yielding 295,712 kWh/day. The 0.40 share is only weakly anchored by 8,800 HDB season-parking EVs versus 21,796 national electric cars in September 2024 [5]. Housing and population weights provide alternatives.

$$q = pHa\rho = 7.4 \times 6 \times 0.97 \times 0.85 = 36.6078$$

$$Q_j^0 = Ha\rho \sum_{r \in j} \max_{k \in r} \{p_k\}$$

Registration maximum-port power is the conservative bound; summed port power is a sensitivity.

Effective hours, uptime and accepted-power factors are design assumptions. The public-carpark average of about five hours/day in 2026 does not measure HDB-only delivered energy [6]. Charging losses enter demand once; the power factor represents accepted charging rate. Capacity does not guarantee short queues.

3. Model and methodology

The facility-location model combines integer investment and continuous energy flows [8,9]. HDB carparks are both demand nodes i and candidate sites j. Catchment arcs may cross planning-area boundaries.

Symbol	Meaning
x_j; z_j; y_j	AC points; activation binary; upgrade binary
f_{ij}; s_i	Served and unmet grid-side kWh/day
Q^o_j; q	Existing and per-new-point effective daily capacity
M_j; $\bar{L}_j$; b^o_j	Addition bound; total bay limit; existing occupied bays

Constraints

$$\sum_{j:(i,j)\in A} f_{ij} + s_i = D_i, \qquad f_{ij}, s_i \geq 0$$

$$\sum_{i:(i,j)\in A} f_{ij} \leq Q_j^0 + qx_j$$

$$x_j \leq M_j z_j, \quad z_j \leq x_j, \quad y_j \leq z_j$$

$$7.4x_j \leq G_j + U_j y_j, \quad x_j + b_j^0 \leq \bar{L}_j$$

$$\sum_j (c_j x_j + F_j z_j + K_j y_j) \leq B$$

$$x_j \in \mathbb{Z}_+, \quad y_j, z_j \in \{0, 1\}$$

Objectives and annual cost

$$S^* = \max \sum_{(i,j)\in A} f_{ij}$$

$$\min C_{ann} \quad \text{s. t.} \quad \sum_{(i,j)\in A} f_{ij} \geq S^* - \varepsilon$$

$$C_{cap} = \sum_j (c_j x_j + F_j z_j + K_j y_j)$$

$$C_{ann} = \frac{r}{1-(1+r)^{-n}} C_{cap} + \sum_j (m_j + v_j) x_j$$

$$\text{Cost/extra annual kWh} = \frac{C_{ann}}{365(S - S_0)}$$

Constraints conserve demand and enforce capacity, activation, power, space and budget. The bay limit is max(existing bays, floor(0.10 × lots)); existing bays are grandfathered. One connector per existing bay is tested against deduplicated bay labels.

Unmet demand preserves feasibility; s is implemented as demand minus outgoing flow. Spare headroom is assumed.

3.1 Objectives, cost and numerical checks

Pass 1 maximises served energy. Pass 2 minimises annual infrastructure cost while retaining service within ε = 0.002957 kWh/day. A subsequent fixed-investment LP minimises energy-weighted travel distance. Objectives are sequential and deterministic.

Costs [7]: S$6,500/AC point including ordinary installation; S$580/point/year maintenance and network reserves; S$50,000/site upgrade. Capital uses a 10-year life and 5% rate. Constant 2026 SGD excludes GST, electricity, sunk investment and revenue.

Each point adds at most q kWh/day: at least ceil((target − existing-only service)/q) points are needed. Attaining this bound without upgrades certifies minimum capex. Expected-answer tests cover integrality, upgrades, sharing and infeasibility; national runs check residuals and service locks.

4. Planning-area allocation for end-2027

The 90%-service plan adds 2,332 AC points across 549 carparks in 28 planning areas, at S$15.158m. These are new additions for end-2027 demand, not total installed supply or a 2030 build plan.

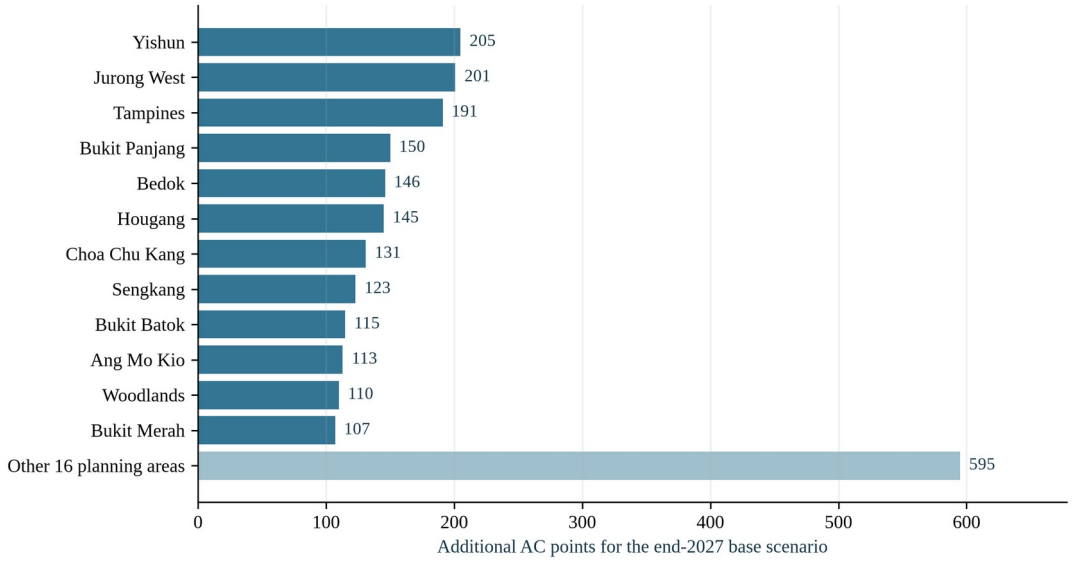


Figure 1. End-2027 base allocation by planning area. The final bar combines 16 areas.

Selected planning areas	Carparks receiving additions	Additional AC points
Yishun	43	205
Jurong West	44	201
Tampines	47	191
Bedok	33	146
Sengkang	30	123
Ang Mo Kio	22	113
Woodlands	27	110

Selected planning areas	Carparks receiving additions	Additional AC points
Punggol	21	84
Other 20 planning areas	282	1159
Total	549	2332

The workbook lists all 28 areas. Site coordinates and verification examples are retained in the appendix and detailed sheets. Imputed-capacity sites receive 11.4% of expenditure and require confirmation.

4.1 Top 100 carparks for site review

The map highlights 100 review candidates within the saved end-2027 plan: 528 proposed AC points across 21 planning areas. The full national plan remains 549 carparks and 2,332 points.

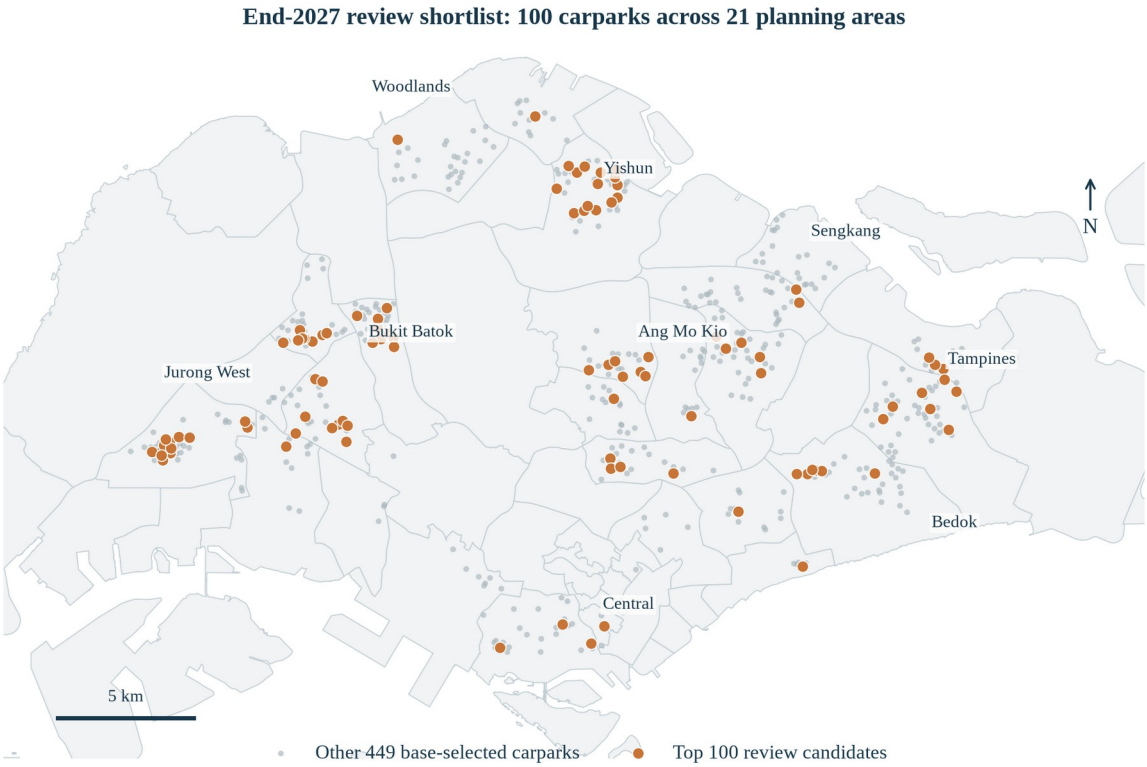


Figure 2. Orange: top 100 ranked carparks. Grey: other 449 selected carparks. Locations use source coordinates transformed to WGS84; boundaries are URA MP2019.

Ranking step	Rule
Eligibility	Base-selected carparks with observed parking capacity and night parking.
Primary order	Selection frequency across 37 scenarios with both objective passes certified, descending.
Ties	More AC additions in the base solution first, then carpark ID alphabetically.
Traceability	Top_100_Sites in Excel gives all 100 ranks, addresses, planning areas, latitude and longitude.

These sites appear in 56.8–81.1% of certified scenarios. Selection frequency measures recurrence under the tested assumptions; it is not a probability or a marginal-benefit score. No geographic quota was applied.

This is a transparent review shortlist, not a separately optimised 100-site deployment. The 90% service result applies to the entire national plan. Existing chargers, local demand, access and electrical headroom still require confirmation at every highlighted site.

4.2 Comparison with simple rules

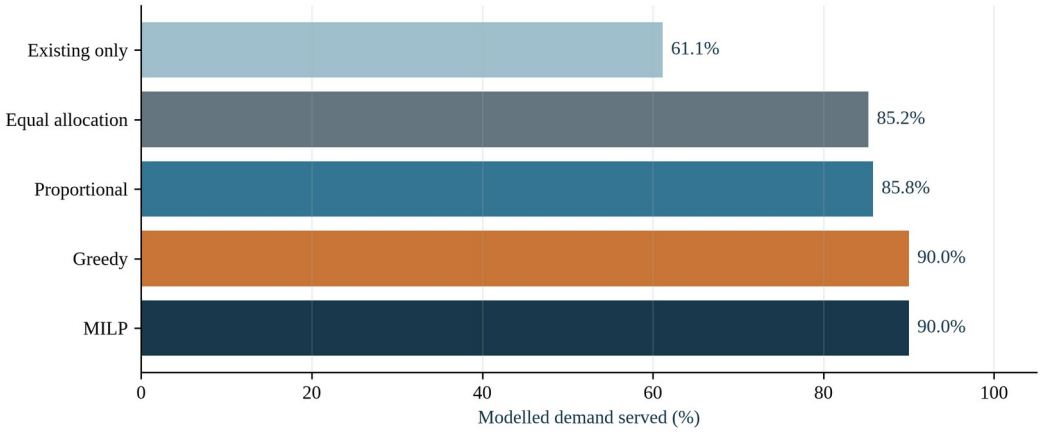


Figure 3. Same modelled demand and feasibility rules. Actual expenditure is reported below.

Allocation	Capital	AC points	Served
Equal	S$15.158m	2,332	85.2%
Proportional	S$15.156m	2,324	85.8%
Greedy	S$15.158m	2,332	90.0%
MILP	S$15.158m	2,332	90.0%

The MILP improves service over proportional placement by 4.19 percentage points. Equal allocation cycles through eligible site IDs; proportional allocation uses largest remainders and

feasibility repair. Neither rule directly accounts for overlapping catchments or unused existing capacity.

Greedy placement chooses additional served energy per step dollar, including upgrade thresholds and full flow reassignment. Its service is 90.0%, versus 90.0% for the MILP. Greedy matches the MILP to numerical tolerance in this base case; the MILP chiefly supplies an optimum certificate here. The workbook also reoptimises each rule's actual spend; time-limited comparison incumbents retain their gaps.

5. Cost-effectiveness and budget choices

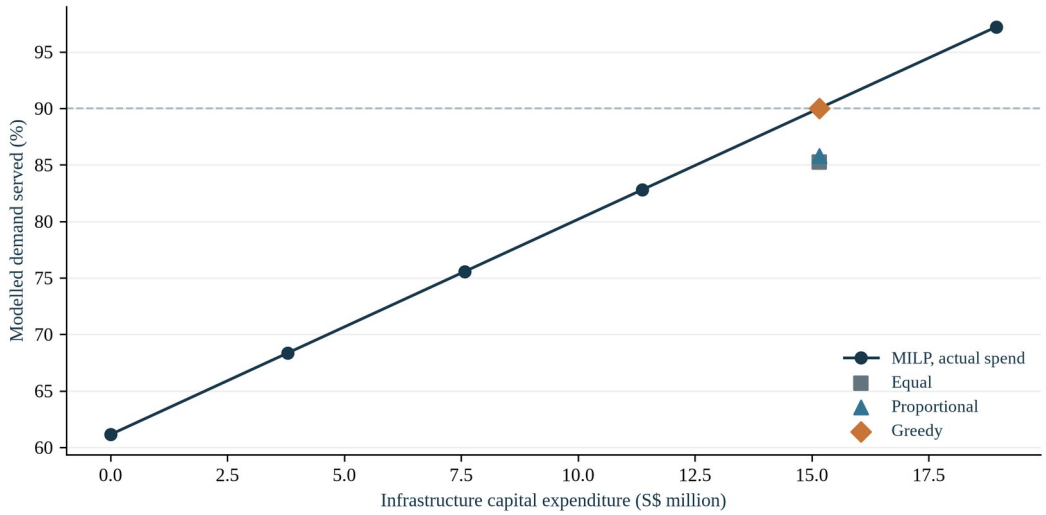


Figure 4. Discrete actual-spend frontier. Points are computed scenarios, not a smooth forecast.

Service target	Minimum capital	Certificate
80%	S$9.912m	Capacity lower bound attained
90%	S$15.158m	Capacity lower bound attained
95%	S$17.784m	Capacity lower bound attained

At the base budget, annualised infrastructure cost is S$3.316m/year, or S$0.106 per additional annual kWh served. This excludes electricity and is not a charging tariff or a social

benefit-cost ratio. It divides incremental annual infrastructure cost by service above the fixed existing-only baseline.

Across the computed budget intervals, marginal capex per extra annual kWh ranges from S$0.486 to S$0.487. A nearly constant slope is possible while each added point remains fully useful; diminishing returns must be observed, not presumed. The physically bounded maximum service is 100.0% under the base assumptions.

6. Sensitivity and fixed-plan performance

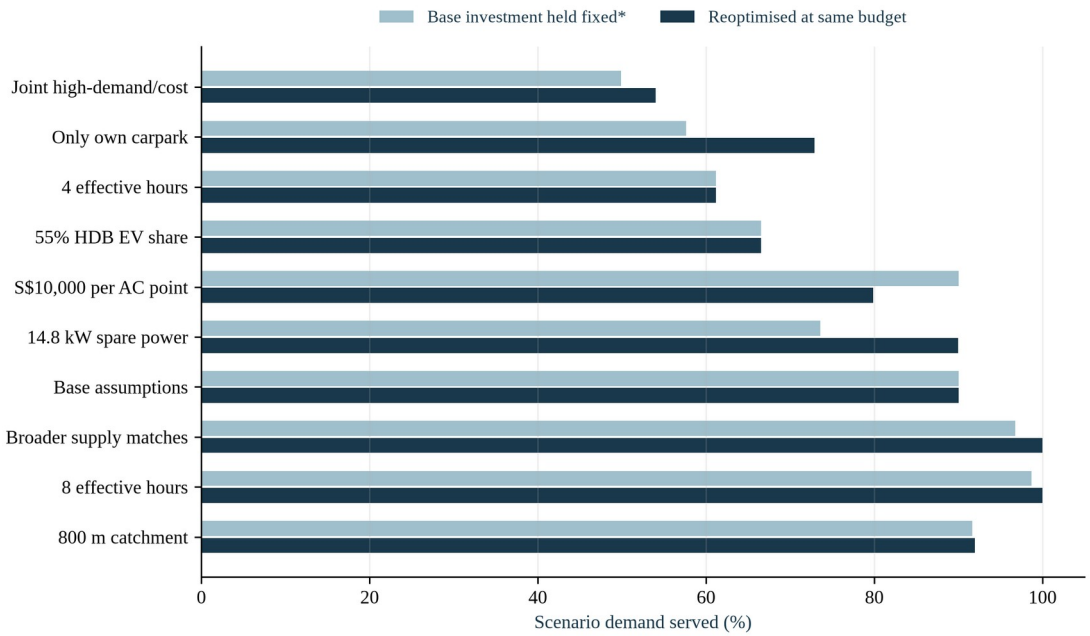


Figure 5. Selected stresses at the base nominal budget. *Fixed-plan capacity is derated where power/space assumptions invalidate operation; repriced capex may exceed budget.

Broader supply matching changes reoptimised service to 100.0%. Four effective charging hours reduce it to 61.2%; restricting service to the home carpark gives 72.9%. These differences show why supply identification, usable hours and access rules must be checked before converting model outputs into construction decisions.

The workbook contains 52 specified stress cases, each with a fixed-plan evaluation and reoptimisation. 5 service solves lack optimal status; bounds and gaps remain visible. Secondary cost gaps can be larger, so stressed annual costs are attained costs, not automatically minimum costs. Feasible fixed-plan regret is bounded using the scenario incumbent and service bound. Scenario frequencies are not probabilities.

6.1 Current and future energy shortfalls

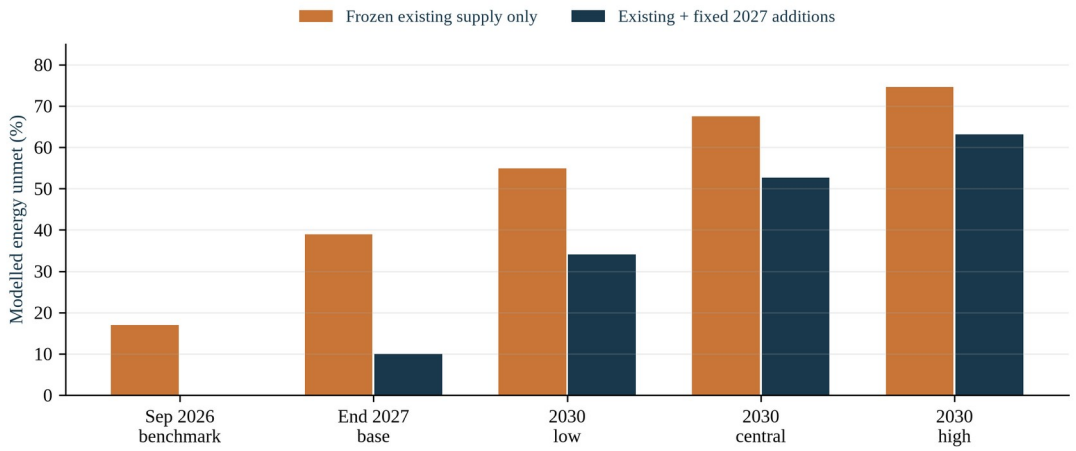


Figure 6. Energy shortfalls under frozen supply and the fixed 2027 additions. Future additions are not shown as installed in 2026.

Demand period	Demand (MWh/day)	Unmet: existing only	Unmet: +2027 plan
Sep 2026 benchmark	189.1	17.0%	—
End 2027 base	295.7	38.9%	10.0%
2030 low	410.9	54.9%	34.1%
2030 central	575.3	67.5%	52.6%
2030 high	739.7	74.7%	63.1%

The 2026 benchmark uses observed September national EV stock but estimated HDB energy demand. The 2027 base extends registration flows for 15 months with assumed exits. The 2030 cases assume 25/35/45% of the frozen car fleet is electric; they are scenarios, not fitted forecasts. Existing supply stays frozen, and the second supply case adds the same 2,332 AC points with no further expansion.

The central 2030 case leaves 67.5% unmet under existing supply, or 52.6% with the 2027 plan. Meeting 90% in 2027 therefore does not establish adequacy in 2030. A separate 2030 target-cost investment plan has not been solved.

The 2027 AC/DC extension selects 0 new DC points and serves 90.0%; the tested costs provide no case for forcing a fast-charging quota.

7. Recommendations and limitations

Use the 28-area allocation and S$15.158m envelope to discuss a conditional end-2027 plan serving 90% of estimated energy demand. Yishun, Jurong West and Tampines receive the largest point allocations. The lower-bound certificate supports the national point count and budget; it does not make each area allocation uniquely optimal.

Before construction	Why it can change the decision
Confirm supply and access	Check existing HDB ownership, shared cabinet output, bay labels, walking routes and parking entitlement.
Measure local demand	Obtain resident EV counts, session energy and occupation patterns; update spatial weights and effective hours.
Survey electrical feasibility	Verify spare headroom and obtain a site-specific upgrade design and installed-cost quotation.

Before construction	Why it can change the decision
Stage and rerun	Prioritise observed-capacity review candidates; verify uncertain sites before committing their allocations. Reoptimise after measured changes.

Keep the three time horizons distinct: the 2026 modelled benchmark, the end-2027 investment decision, and the 2030 demand stress. The proposed plan leaves a deliberate 10% energy gap in 2027 and requires further expansion as demand grows. Update area-level budgets when better local demand and confirmed supply become available.

Individual locations are less stable: alternate seeds produce only 22.9% site-set overlap at near-equivalent service. The five reviewed sites are examples in the appendix, not the national deployment list or a geographically balanced priority ranking.

Demand remains a spatial proxy. Conservative matching can inflate apparent need, and uniform headroom, costs and charging hours cannot replace site evidence. Straight-line access can overstate feasible parking and walking connections.

The current model does not estimate queues, reliability at peak times or adoption causality. Its strongest contribution is a reproducible chain from dated inputs to feasible decisions, comparison rules and sensitivity tests. LP allocation, integer investment, relaxation bounds and finite budget differences supply the relevant analytical foundation; an additional dynamic programme is unnecessary for this static decision.

References

[1] HDB. Carpark Information; carpark availability snapshot, 31 October 2026. Local master and lot files frozen for this analysis. HDB dataset

[2] LTA. Public EV charging point register, September 2026. Unique point IDs and registration-level power bounds. LTA DataMall

[3] LTA. Monthly vehicle population by fuel; new car registrations; annual private vehicle mileage. September 2026 / 2025 vintages. Vehicle statistics

[4] Singapore Department of Statistics. Resident population and dwelling stock by planning area/subzone and dwelling type, 2025 and 2026. URA MP2019/MP2025 boundaries. SingStat geographic distribution

[5] Ministry of Transport. EVs registered for season parking in HDB carparks. Parliamentary reply, 11 November 2024. Historical calibration reply

[6] Ministry of Transport. Utilisation rates of EV charging points at petrol kiosks relative to public carparks. 10 September 2026. Utilisation reply

[7] Electric Pte Ltd. EV charger hardware and maintenance schedule; archived 9 November 2026. Commercial comparator, not an HDB installed-cost contract. Vendor evidence

[8] Gurobi Optimization. Facility-location example and Multiple Objectives documentation, archived 9 November 2026. Structural references; this implementation uses HiGHS. Facility-location example

[9] Church, R. and ReVelle, C. (1974). The maximal covering location problem. Papers of the Regional Science Association, 32, 101–118. Bibliographic context; original full text was not retrieved. DOI

[10] Huangfu, Q. and Hall, J. A. J. (2018). Parallelizing the dual revised simplex method. Mathematical Programming Computation, 10, 119–142. HiGHS solver documentation accessed 10 November 2026. HiGHS documentation

[11] DBA5103 Project Guideline. Semester I, 2026/2027, NUS. Local course document: sources/references/Project_Guideline.pdf.

Full file paths, collection timestamps, source URLs, parameter evidence and SHA-256 hashes are retained in the workbook and local research manifest. All numerical results use frozen local inputs, rather than a live data refresh.

Appendix A. Model audit and reproducibility

Item	Audit result
Model size	2,273 sites; 21,003 allowable service arcs; continuous flows plus integer and binary investment decisions.
Source points	11,489 unique IDs; 5,342 address-supported; remaining classes stay separate.
Capacity imputation	278 sites; 71,123.5 lots. Base total 777,786.5.
Solver	HiGHS 1.15.1; Python 3.14.4; one thread; seed 17.
Tolerances	Base relative MIP gap 0.0001; stresses 0.001. Feasibility tolerance 1e-7. Service lock max(1e-6, 1e-8 × total demand).
Runtime limits	Base 45 seconds per MILP pass; sensitivity 25 seconds. Actual elapsed times and overshoot appear in logs.
Repeatability	Identical-input repeat: additions identical = True; service difference 0.000000 kWh/day.
Artifacts	HDB_EV_Charging_Analysis.xlsx; code/outputs/runs, solver_logs, inputs, figures; code/requirements.txt; code/venv/.

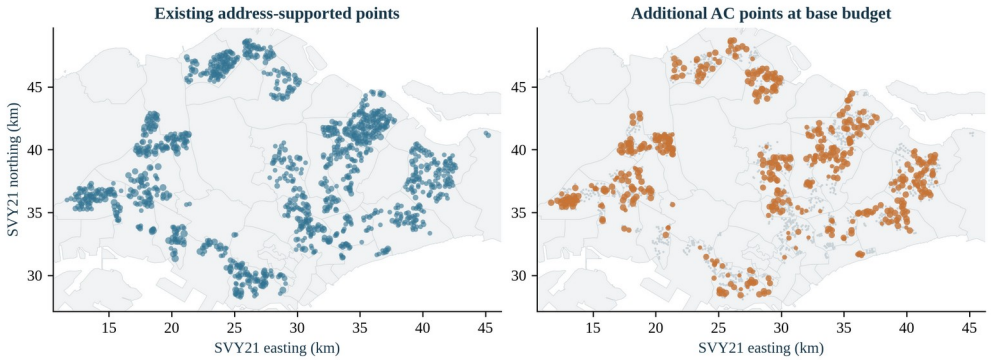
Space and power residuals, demand conservation, nonnegative flows, cost reconciliation, service locks and integrality are checked before saving integer results. A mathematical lower bound certifies the service-target budgets when attained. An LP relaxation provides an upper bound on service, but its fractional decisions are not installable. Marginal investment comparisons use finite differences, not MILP shadow prices.

Fixed-plan stress holds investment and upgrade binaries constant. If the scenario allows fewer bays or less power, usable added energy is derated by the tighter proportional limit, while sunk point counts remain unchanged. Such plans are flagged as construction-infeasible; repriced plans can also exceed the scenario budget. Their difference from a same-budget optimum is not labelled feasible-plan regret.

The 2027 stock path assumes the Jan–Sep 2026 mean registration flow persists for 15 months; exits are a stated fraction of these additions. It is not a fitted forecast, so no statistical confidence interval or forecasting back-test is claimed. Existing DC uses the same declared effective-hours base while retaining recorded power.

Appendix A.1 Site-level verification examples

The main recommendation is the 28-area allocation. The five examples below were chosen by scenario selection frequency among base-selected sites with observed capacity and night parking; no geographical-diversity constraint was applied. Their western concentration does not represent the island-wide deployment.



Appendix figure. Existing matched supply and all 549 selected carparks; not only the five review examples.

Review example	Added AC	Source-record review gate
JM18 · Jurong West	6	Source-supported; electrical/access survey
U15 · Bukit Batok	1	HOLD: resolve six provisional points
J98M · Jurong West	6	HOLD: resolve duplicate-address supply
JM13 · Jurong West	6	Source-supported; electrical/access survey
U13 · Bukit Batok	6	HOLD: resolve nearby address mismatch

Frequencies use 37 certified stress cases. Supply uncertainties remain explicit holds; all locations require electrical and access surveys. Equal-budget replacement solves recover nearly all service. Full addresses, coordinates, quantities and review findings remain in the workbook.

Appendix B. Execution and AI-use declaration

From the repository root, use the commands below. The environment already exists in code/venv/; requirements.txt pins the installed libraries. LibreOffice is used only to render the DOCX to a checked PDF and is a system dependency.

```
python3 -m venv project/code/venv
project/code/venv/bin/python -m pip install -r project/code/requirements.txt
project/code/venv/bin/python project/code/prepare_data.py
project/code/venv/bin/python project/code/test_models.py
project/code/venv/bin/python project/code/run_analysis.py --mode all
project/code/venv/bin/python project/code/review_shortlist.py
project/code/venv/bin/python project/code/audit_outputs.py
project/code/venv/bin/python project/code/build_figures.py
project/code/venv/bin/python project/code/build_workbook.py
project/code/venv/bin/python project/code/build_report.py
```

AI assistance: OpenAI Codex assisted with Python implementation, analysis checks, drafting and document formatting. No external party was contacted and no site verification was performed. The submitting authors remain responsible for reviewing the assumptions, evidence and final submission.

The following appendix attaches the complete data preparation, model, comparison, scenario and test code. Office-document and plotting generators are also supplied as Python files in code/. Machine-readable outputs and solver logs accompany the report rather than being repeated as long printed tables.

Appendix C. Python — prepare_data.py

```
"""Freeze auditable HDB inputs from the supplied local source vintage."""
from pathlib import Path
import hashlib
import json
import re
import numpy as np
import pandas as pd
from pyproj import Transformer
from scipy.spatial import cKDTree
from shapely.geometry import Point, shape
from shapely.strtree import STRtree

ROOT = Path(__file__).resolve().parents[1]
DATA = ROOT / 'data'
OUT = Path(__file__).resolve().parent / 'outputs'
INPUTS = OUT / 'inputs'
USED = set()


def read(rel, **kw):
    p = DATA / rel
    USED.add(p)
    return pd.read_csv(p, **kw)


def normal(text):
    s = str(text).upper() if pd.notna(text) else ''
    for a, b in {'AVE': 'AVENUE', 'ST': 'STREET', 'RD': 'ROAD',
                 'DR': 'DRIVE', 'CRES': 'CRESCENT', 'CTRL': 'CENTRAL',
                 'BT': 'BUKIT', 'NTH': 'NORTH', 'STH': 'SOUTH',
                 'UPP': 'UPPER', 'TG': 'TANJONG', 'JLN': 'JALAN',
                 'CL': 'CLOSE', 'PL': 'PLACE', 'TER': 'TERRACE',
                 'KG': 'KAMPONG', 'LOR': 'LORONG', 'GDNS': 'GARDENS'}.items():
        s = re.sub(r'\b' + a + r'\b', b, s)
    return ' '.join(re.sub(r'[^A-Z0-9 ]', ' ', s).split())


def address_parts(address):
    s = str(address).upper()
    m = re.match(r'(?:BLKS?\s*)?(\d+[A-Z]?(?:\s*[/,&-]\s*\d+[A-Z]?)*)\s+([A-Z].*)', s)
    if not m:
        return set(), '', []
    blocks = set(re.findall(r'\d+[A-Z]?', m[1]))
    ranges = []
    for a, b in re.findall(r'(\d+)\s*[/\-]\s*(\d+)(?![A-Z])', m[1]):
        if 0 <= int(b) - int(a) <= 60:
            ranges.append((int(a), int(b)))
    street = re.sub(r'\b(?:MULTI.?STOREY|BASEMENT|SURFACE|CAR PARK|CARPARK|MSCP).*', '', m[2]).strip()
    return blocks, normal(street), ranges


def geotag(sites, year):
    p = DATA / f'05_geospatial_boundaries/URA_MP{year}_planning_area_boundary.geojson'
    USED.add(p)
    fs = json.loads(p.read_text())['features']
    gs = [shape(f['geometry']) for f in fs]
    tree = STRtree(gs)
    names = []
    for lon, lat in zip(sites.longitude, sites.latitude):
        pt = Point(lon, lat)
        hits = [int(k) for k in tree.query(pt) if gs[k].covers(pt)]
        names.append(fs[hits[0]]['properties']['PLN_AREA_N'] if hits else 'UNMAPPED')
    sites[f'area_{year}'] = names


def build_sites():
    s =
read('03_carparks_candidate_sites/HDB_carpark_information.csv').sort_values('car_park_no').reset_index(drop
=True)
    l = read('03_carparks_candidate_sites/HDB_carpark_lots_snapshot_2026-10-31.csv')
    l = l[l.lot_type.eq('C')]
    assert s.car_park_no.is_unique and l.car_park_no.is_unique
    s = s.merge(l[['car_park_no', 'total_lots']], how='left', validate='one_to_one')
    s['capacity_status'] = np.select([s.total_lots.isna(), s.total_lots.eq(0)], ['missing', 'zero'],
default='observed')
    s['imputed'] = s.capacity_status.ne('observed')
    tr = Transformer.from_crs(3414, 4326, always_xy=True)
    s['longitude'], s['latitude'] = tr.transform(s.x_coord.to_numpy(), s.y_coord.to_numpy())
    for y in (2019, 2025):
        geotag(s, y)
    observed = s[~s.imputed]
```

```
    s['lots'] = s.total_lots.copy()
    s['imputation_basis'] = 'observed'
    for i in s.index[s.imputed]:
        r = s.loc[i]
        group = observed[(observed.area_2019 == r.area_2019) & (observed.car_park_type ==
r.car_park_type)]
        basis = 'area/type median'
        if len(group) < 5:
            group = observed[observed.car_park_type == r.car_park_type]
            basis = 'type median'
        if len(group) < 5:
            group, basis = observed, 'national median'
        s.loc[i, 'lots'] = float(group.total_lots.median())
        s.loc[i, 'imputation_basis'] = basis
    s['residential_review_flag'] = s.night_parking.ne('YES')
    # All HDB sites remain in the agreed base scope; night-parking is a review flag.
    assert len(s) == 2273 and s.lots.gt(0).all()
    s.to_csv(INPUTS / 'sites.csv', index=False)
    s[s.imputed | s.residential_review_flag].to_csv(INPUTS / 'site_exceptions.csv', index=False)
    return s


 def build_crosswalk(s):
    cp = read('01_ev_charging_supply/LTA_EV_charging_points_Sep2026.csv', dtype={'PostalCode': str,
'Lot No': str, 'Floor No': str, 'Block/House No': str})
    assert cp.evCpId.is_unique and len(cp) == 11489
    cp['PostalCode'] = cp.PostalCode.str.zfill(6)
    tr = Transformer.from_crs(4326, 3414, always_xy=True)
    xy = np.column_stack(tr.transform(cp.longitude.to_numpy(), cp.latitude.to_numpy()))
    tree = cKDTree(s[['x_coord', 'y_coord']])
    parsed = [address_parts(a) for a in s.address]
    excluded = re.compile(r'\b(MALL|PETROL|SHELL|ESSO|CALTEX|SPC|HOSPITAL|HOTEL|CONDOMINIUM|
UNIVERSITY|POLYTECHNIC|INDUSTRIAL|BUSINESS|JTC|IKEA|MUSEUM|AIRPORT|SHOPPING|SCHOOL|PLAZA|COMMUNITY CLUB|
COMMUNITY CENTRE)\b')
    records = []
    # Explicit address evidence outranks nearest distance. Range/near-street matches stay provisional.
    for i, r in cp.iterrows():
        candidates = tree.query_ball_point(xy[i], 150)
        block, street, building = str(r['Block/House No']).upper(), normal(r['Street Name']),
normal(r['Building Name'])
        exact, ranged, near = [], [], []
        for j in candidates:
            blocks, st, ranges = parsed[j]
            if st != street:
                continue
            dist = float(np.linalg.norm(xy[i] - s.loc[j, ['x_coord',
'y_coord']].to_numpy(dtype=float)))
            near.append((dist, j))
            if block in blocks:
                exact.append((dist, j))
            elif block.isdigit() and any(a <= int(block) <= b for a, b in ranges):
                ranged.append((dist, j))
        status, pick, reason = 'unresolved', None, 'No unique exact block/street match within 150 m'
        if excluded.search(building):
            status, reason = 'excluded_non_hdb_evidence', 'Building name indicates non-HDB use;
conservative exclusion'
        elif len(exact) == 1:
            status, pick, reason = 'address_supported', exact[0], 'Unique literal block + normalised
street + distance <=150 m'
        elif len(exact) > 1:
            reason = 'Multiple exact address candidates; no base assignment'
        elif len(ranged) == 1:
            status, pick, reason = 'provisional_range', ranged[0], 'Block lies in short master-address
range; sensitivity only'
        elif len(near) == 1 and near[0][0] <= 100:
            status, pick, reason = 'provisional_street', near[0], 'Unique same-street candidate within
100 m; sensitivity only'
        elif not candidates:
            status, reason = 'outside_candidate_radius', 'No HDB candidate within 150 m'
        records.append({'evCpId': r.evCpId, 'match_status': status,
                        'site_id': s.loc[pick[1], 'car_park_no'] if pick else '',
                        'site_index': pick[1] if pick else np.nan,
                        'match_distance_m': pick[0] if pick else np.nan,
                        'nearby_candidates_50m': len(tree.query_ball_point(xy[i], 50)),
                        'nearby_candidates_100m': len(tree.query_ball_point(xy[i], 100)),
                        'nearby_candidates_150m': len(candidates), 'match_reason': reason})
    c = cp.merge(pd.DataFrame(records), on='evCpId', validate='one_to_one')
    # A registration must not split across sites; unresolved members exclude that registration from
base.
    for _, g in c.groupby('EV Charger Registration Code'):
        assigned = g.site_id[g.site_id.ne('')].unique()
        if len(assigned) > 1 or (g.match_status.eq('address_supported').any() and not
g.match_status.eq('address_supported').all()):
            c.loc[g.index, 'match_status'] = 'registration_conflict'
            c.loc[g.index, ['site_id', 'site_index']] = ['', np.nan]
    c.to_csv(INPUTS / 'hdb_charger_crosswalk.csv', index=False)
    loc = c.groupby(['longitude', 'latitude', 'match_status', 'site_id'], dropna=False).agg(
```

```
154          point_count=('evCpId', 'size'), block=('Block/House No', 'first'), street=('Street Name',
'first'),
155          building=('Building Name', 'first'), distance_m=('match_distance_m', 'first')).reset_index()
156      loc.to_csv(INPUTS / 'location_review_queue.csv', index=False)
157      supply = pd.DataFrame({'site_id': s.car_park_no})
158      for label, statuses in [('strict', ['address_supported']), ('broad', ['address_supported',
'provisional_range', 'provisional_street'])]:
159          subset = c[c.match_status.isin(statuses)].copy()
160          subset['power'] = pd.to_numeric(subset['charging Speed'], errors='raise')
161          assert subset.power.ge(0).all()
162          subset['is_dc'] = ~subset.plugType.eq('Type 2')
163          reg = subset.groupby(['site_id', 'EV Charger Registration Code']).agg(
164              power_lower=('power', 'max'), power_upper=('power', 'sum'), points=('evCpId', 'size'),
dc=('is_dc', 'max')).reset_index()
165          reg['scope'] = label
166          reg.to_csv(INPUTS / f'registration_capacity_{label}.csv', index=False)
167          by = reg.groupby('site_id').agg(kw_lower=('power_lower', 'sum'), kw_upper=('power_upper',
'sum'),
168                                          points=('points', 'sum'), dc_registrations=('dc', 'sum'))
169          for col in by:
170              supply[f'{label}_{col}'] = supply.site_id.map(by[col]).fillna(0)
171      # Distinct floor + single lot is an alternative bay estimate; multi-lot/absent labels use points.
172      sub = c[c.match_status.eq('address_supported')].copy()
173      sub['bay_key'] = [f'{fl}:{lot}' if re.fullmatch(r'\d+[A-Z]?', str(lot)) and pd.notna(fl)
174                        else f'unknown:{pid}' for fl, lot, pid in zip(sub['Floor No'], sub['Lot No'],
sub.evCpId)]
175      bays = sub.groupby('site_id').bay_key.nunique()
176      supply['strict_bays_deduplicated'] = supply.site_id.map(bays).fillna(0)
177      supply.to_csv(INPUTS / 'existing_supply.csv', index=False)
178      return c, supply
179
180
181  def build_demographics():
182      frames = []
183      for year in (2025, 2026):
184          for kind, value in [('population', 'Pop'), ('dwellings', 'HSE')]:
185              stem = f'population_dwelling_{year}__respopagesextod{year}.csv' if kind == 'population'
else f'dwellings_type_{year}__hsetod{year}.csv'
186              d = read('04_demographics_demand_proxies/updates_2026-11-09/' + stem)
187              assert value in d, (stem, list(d.columns))
188              d = d[d.TOD.str.startswith('HDB ')].copy()
189              d[value] = pd.to_numeric(d[value], errors='raise')
190              a = d.groupby('PA', as_index=False)[value].sum().rename(columns={value: 'value'})
191              a['area'] = a.PA.str.upper()
192              a['year'], a['metric'], a['source_file'] = year, kind, stem
193              frames.append(a)
194      a = pd.concat(frames, ignore_index=True)
195      a.to_csv(INPUTS / 'area_demand_weights.csv', index=False)
196      return a
197
198
199  def build_stock():
200      v = read('02_ev_population_and_registrations/LTA_M09_monthly_vehicle_population_by_fuel.csv')
201      v = v[v.category.eq('Cars')].copy()
202      v['numeric'] = pd.to_numeric(v.number, errors='coerce')
203      v[v.numeric.isna()].to_csv(INPUTS / 'vehicle_numeric_exceptions.csv', index=False)
204      ev = v[v.type.eq('Electric')].set_index('month').numeric
205      fleet = v.groupby('month').numeric.sum(min_count=1)
206      r =
read('02_ev_population_and_registrations/LTA_M03_monthly_new_car_registrations_by_make_fuel.csv')
207      r['numeric'] = pd.to_numeric(r.number, errors='coerce')
208      reg = r[r.fuel_type.eq('Electric')].groupby('month').numeric.sum(min_count=1)
209      hist = pd.DataFrame({'ev_stock': ev, 'all_car_stock': fleet, 'new_electric_cars':
reg}).sort_index()
210      hist['implied_net_exits'] = hist.ev_stock.shift() + hist.new_electric_cars - hist.ev_stock
211      hist.to_csv(INPUTS / 'ev_history.csv', index_label='month')
212      last = hist.loc['2026-09']
213      monthly = hist.loc['2026-01':'2026-09', 'new_electric_cars'].mean()
214      rows = [{'scenario': 'current_2026', 'year': 2026, 'stock': last.ev_stock, 'basis': 'Observed
September 2026 Cars/Electric', 'monthly_registrations': np.nan, 'exit_fraction': np.nan}]
215      for name, factor, exits in [('low', .75, .05), ('base', 1., .02), ('high', 1.25, 0.)]:
216          stock = min(last.all_car_stock, last.ev_stock + 15 * monthly * factor * (1 - exits))
217          rows.append({'scenario': f'2027_{name}', 'year': 2027, 'stock': stock, 'basis': '15 months of
assumed registrations less exits; not fitted forecast', 'monthly_registrations': monthly * factor,
'exit_fraction': exits})
218      for name, share in [('low', .25), ('base', .35), ('high', .45)]:
219          rows.append({'scenario': f'2030_{name}', 'year': 2030, 'stock': last.all_car_stock * share,
'basis': f'{share:.0%} of frozen Sep-2026 all-car stock; illustrative stress case',
'monthly_registrations': np.nan, 'exit_fraction': np.nan})
220      scenarios = pd.DataFrame(rows)
221      scenarios.to_csv(INPUTS / 'ev_stock_scenarios.csv', index=False)
222      # Historical anchor consistency, not a validation of spatial demand.
223      anchor = {'sep2024_electric_cars': float(ev.loc['2024-09']), 'hdb_season_parking_evs': 8800,
224                'historical_hdb_ratio': float(8800 / ev.loc['2024-09']),
225                'stock_path_kind': 'assumed scenarios; no fitted extrapolation or confidence interval',
226                '2026_registration_missing_rows': int(r[(r.month >= '2026-01') &
r.fuel_type.eq('Electric')].numeric.isna().sum())}
```

```
    (INPUTS / 'stock_calibration.json').write_text(json.dumps(anchor, indent=2))
    return scenarios


def main():
    INPUTS.mkdir(parents=True, exist_ok=True)
    s = build_sites()
    c, supply = build_crosswalk(s)
    build_demographics()
    scenarios = build_stock()
    read('08_costs/cost_parameters.csv').to_csv(INPUTS / 'parameter_evidence.csv', index=False)
    manifest = [{'path': str(p.relative_to(ROOT)), 'sha256':
hashlib.sha256(p.read_bytes()).hexdigest(), 'bytes': p.stat().st_size} for p in sorted(USED)]
    (INPUTS / 'source_hashes.json').write_text(json.dumps(manifest, indent=2))
    audit = {'sites': len(s), 'missing_capacity': int(s.capacity_status.eq('missing').sum()),
             'zero_capacity': int(s.capacity_status.eq('zero').sum()), 'imputed_lots':
float(s.loc[s.imputed, 'lots'].sum()),
             'observed_lots': float(s.total_lots.sum()), 'total_lots_after_imputation':
float(s.lots.sum()),
             'charger_points': len(c), 'match_counts': c.match_status.value_counts().to_dict(),
             'strict_sites': int(supply.strict_points.gt(0).sum()),
             'strict_dc_registrations': int(supply.strict_dc_registrations.sum()),
             'unmapped_hdb_sites': int(s.area_2019.eq('UNMAPPED').sum())}
    (INPUTS / 'data_audit.json').write_text(json.dumps(audit, indent=2))
    print(json.dumps(audit, indent=2), flush=True)
    print(scenarios.to_string(index=False), flush=True)


if __name__ == '__main__':
    main()
```

Appendix C. Python — model.py

```
"""Capacitated location MILP and fixed-investment transportation LP (HiGHS)."""
from dataclasses import dataclass, replace, asdict
from pathlib import Path
import json
import time
import numpy as np
import pandas as pd
import highspy as hs
from scipy.sparse import coo_matrix
from scipy.spatial import cKDTree
from prepare_data import INPUTS, OUT


@dataclass(frozen=True)
class Parameters:
    stock: float = 117116.83333333333
    hdb_share: float = .4
    dependence: float = .8
    annual_km: float = 16200
    intensity: float = .16
    efficiency: float = .9
    hours: float = 6
    availability: float = .97
    power_factor: float = .85
    radius: float = 400
    path_inflation: float = 1
    ac_power: float = 7.4
    dc_power: float = 50
    dc_hours: float = 6
    dc_power_factor: float = .85
    ac_cost: float = 6500
    dc_cost: float = 60000
    fixed_cost: float = 0
    upgrade_cost: float = 50000
    ac_maintenance: float = 380
    dc_maintenance: float = 1180
    network: float = 200
    lifetime: float = 10
    discount: float = .05
    headroom: float = 44.4
    upgrade_kw: float = 88.8
    lot_share: float = .1
    impute_factor: float = 1
    unknown_no_build: bool = False
    weights: str = 'lots'
    matching: str = 'strict'
    power_bound: str = 'lower'
    deduplicate_bays: bool = False
    dc_enabled: bool = False
    seed: int = 17
    time_limit: float = 45
    mip_gap: float = .0001

    @property
    def q(self):
        return self.ac_power * self.hours * self.availability * self.power_factor

    @property
    def qdc(self):
        return self.dc_power * self.dc_hours * self.availability * self.dc_power_factor

    @property
    def crf(self):
        return self.discount / (1 - (1 + self.discount) ** -self.lifetime) if self.discount else 1 /
self.lifetime


@dataclass
class Instance:
    sites: pd.DataFrame
    p: Parameters
    demand: np.ndarray
    q0: np.ndarray
    bays: np.ndarray
    maximum: np.ndarray
    ii: np.ndarray
    jj: np.ndarray
    distances: np.ndarray
    unallocated_proxy_share: float = 0

```

```
080      @property
081      def n(self):
082          return len(self.demand)
083
084
085  def build_instance(p=None):
086      p = p or Parameters()
087      s = pd.read_csv(INPUTS / 'sites.csv')
088      supply = pd.read_csv(INPUTS / 'existing_supply.csv')
089      assert s.car_park_no.tolist() == supply.site_id.tolist()
090      lots = s.lots.to_numpy().copy()
091      lots[s.imputed] *= p.impute_factor
092      weights = lots / lots.sum()
093      missing_share = 0.
094      if p.weights != 'lots':
095          a = pd.read_csv(INPUTS / 'area_demand_weights.csv')
096          year = 2026 if p.weights.endswith('2026') else 2025
097          metric = 'population' if p.weights.startswith('population') else 'dwellings'
098          a = a[(a.year == year) & (a.metric == metric)].set_index('area').value
099          area = s['area_2025' if year == 2026 else 'area_2019']
100          area_lots = pd.Series(lots).groupby(area).sum()
101          missing_share = float(a.loc[~a.index.isin(area_lots.index)].sum() / a.sum())
102          # Preserve the national HDB total and disclose any no-site area proxy reallocation.
103          available = a.reindex(area_lots.index).fillna(0)
104          weights = (area.map(available / available.sum()).to_numpy() * lots /
area.map(area_lots).to_numpy())
105      per_ev = p.dependence * p.annual_km / 365 * p.intensity / p.efficiency
106      demand = p.stock * p.hdb_share * per_ev * weights
107      q0 = supply[f'{p.matching}_kw_{p.power_bound}'].to_numpy() * p.hours * p.availability *
p.power_factor
108      # Existing DC retains its recorded rated power under the same declared effective-hours base.
109      bays = supply[f'{p.matching}_points'].to_numpy()
110      if p.deduplicate_bays and p.matching == 'strict':
111          bays = supply.strict_bays_deduplicated.to_numpy()
112      maximum = np.maximum(0, np.floor(p.lot_share * lots) - bays).astype(int)
113      if p.unknown_no_build:
114          maximum[s.imputed] = 0
115      coords = s[['x_coord', 'y_coord']].to_numpy()
116      tree = cKDTree(coords)
117      arcs = [(i, j) for i, hits in enumerate(tree.query_ball_tree(tree, p.radius / p.path_inflation))
for j in sorted(hits)]
118      ii, jj = np.array(arcs, dtype=int).T
119      dist = np.linalg.norm(coords[ii] - coords[jj], axis=1) * p.path_inflation
120      assert np.isclose(weights.sum(), 1) and np.all(demand >= 0)
121      return Instance(s, p, demand, q0, bays, maximum, ii, jj, dist, missing_share)
122
123
124  def make_solver(c, A, lower, upper, lb, ub, integer=None, p=None, log=None):
125      p = p or Parameters()
126      A = A.tocsc()
127      lp = hs.HighsLp()
128      lp.num_col_, lp.num_row_ = len(c), len(lower)
129      lp.col_cost_, lp.col_lower_, lp.col_upper_ = c, lb, ub
130      lp.row_lower_, lp.row_upper_ = lower, upper
131      lp.a_matrix_.format_ = hs.MatrixFormat.kColwise
132      lp.a_matrix_.start_, lp.a_matrix_.index_, lp.a_matrix_.value_ = A.indptr, A.indices, A.data
133      if integer is not None:
134          lp.integrality_ = [hs.HighsVarType.kInteger if t else hs.HighsVarType.kContinuous for t in
integer]
135      h = hs.Highs()
136      h.setOptionValue('output_flag', bool(log))
137      h.setOptionValue('log_to_console', False)
138      if log:
139          Path(log).parent.mkdir(parents=True, exist_ok=True)
140          h.setOptionValue('log_file', str(log))
141      h.setOptionValue('threads', 1)
142      h.setOptionValue('random_seed', p.seed)
143      h.setOptionValue('time_limit', p.time_limit)
144      h._per_solve_seconds = p.time_limit
145      h.setOptionValue('mip_rel_gap', p.mip_gap)
146      h.setOptionValue('mip_feasibility_tolerance', 1e-7)
147      h.setOptionValue('primal_feasibility_tolerance', 1e-7)
148      assert h.passModel(lp) == hs.HighsStatus.kOk
149      return h
150
151
152  def solve_info(h, stage, elapsed):
153      info = h.getInfo()
154      return {'stage': stage, 'status': h.modelStatusToString(h.getModelStatus()),
155              'objective': h.getObjectiveValue(), 'bound': info.mip_dual_bound,
156              'gap': info.mip_gap, 'nodes': info.mip_node_count, 'seconds': elapsed,
157              'max_primal_infeasibility': info.max_primal_infeasibility}
158
159
160  def run(h, stage):
161      # HiGHS run time accumulates when one solver is reused. Give each new solve
162      # its own limit, especially for the thousands of warm-started greedy LPs.
```

```
    h.setOptionValue('time_limit', h.getRunTime() + h._per_solve_seconds)
    start = time.perf_counter()
    h.run()
    info = solve_info(h, stage, time.perf_counter() - start)
    if not h.getSolution().value_valid:
        return None, info
    return np.array(h.getSolution().col_value), info


def investments(inst, x, d=None):
    p = inst.p
    d = np.zeros(inst.n) if d is None else d
    z = (x + d > 0).astype(int)
    y = (p.ac_power * x + p.dc_power * d > p.headroom + 1e-7).astype(int)
    capital = p.ac_cost * x + p.dc_cost * d + p.fixed_cost * z + p.upgrade_cost * y
    annual = p.crf * capital + (p.ac_maintenance + p.network) * x + (p.dc_maintenance + p.network) * d
    return z, y, capital, annual


def allocate(inst, x=None, d=None, distance=True, q_override=None):
    """Max-flow LP, followed by min-distance flow with service preserved."""
    n, m = inst.n, len(inst.ii)
    x = np.zeros(n) if x is None else np.array(x)
    d = np.zeros(n) if d is None else np.array(d)
    q = inst.q0 + inst.p.q * x + inst.p.qdc * d if q_override is None else q_override
    rows = np.r_[inst.ii, n + inst.jj]
    cols = np.tile(np.arange(m), 2)
    A = coo_matrix((np.ones(2*m), (rows, cols)), shape=(2*n, m))
    h = make_solver(-np.ones(m), A, np.zeros(2*n), np.r_[inst.demand, q], np.zeros(m), np.full(m,
np.inf), p=inst.p)
    sol, info = run(h, 'allocation_max_service')
    if sol is None:
        raise RuntimeError(info)
    served = sol.sum()
    if distance and served > 0:
        epsilon = max(1e-6, 1e-8 * inst.demand.sum())
        h.addRow(served - epsilon, np.inf, m, np.arange(m, dtype=np.int32), np.ones(m))
        h.changeColsCost(m, np.arange(m, dtype=np.int32), inst.distances / 1000)
        candidate, dist_info = run(h, 'allocation_min_distance')
        if candidate is not None:
            sol = candidate
    return sol, info


def formulation(inst, budget=np.inf, target=None, relaxation=False):
    """Column blocks f, x_AC, x_DC, z, y; unmet demand eliminated as D - sum(f)."""
    p, n, m = inst.p, inst.n, len(inst.ii)
    nv = m + 4*n
    ix, idc, iz, iy = [m + k*n + np.arange(n) for k in range(4)]
    rr, cc, vv, lower, upper = [], [], [], [], []

    def row(cols, vals, lo=-np.inf, hi=np.inf):
        k = len(lower)
        rr.extend([k]*len(cols)); cc.extend(cols); vv.extend(vals)
        lower.append(lo); upper.append(hi)

    by_i = [[] for _ in range(n)]; by_j = [[] for _ in range(n)]
    for a, (i, j) in enumerate(zip(inst.ii, inst.jj)):
        by_i[i].append(a); by_j[j].append(a)
    for i in range(n):
        row(by_i[i], [1]*len(by_i[i]), hi=inst.demand[i])
    for j in range(n):
        row(by_j[j] + [ix[j], idc[j]], [1]*len(by_j[j]) + [-p.q, -p.qdc], hi=inst.q0[j])
        row([ix[j], idc[j], iz[j]], [1, 1, -inst.maximum[j]], hi=0)
        row([iz[j], ix[j], idc[j]], [1, -1, -1], hi=0)
        row([ix[j], idc[j], iy[j]], [p.ac_power, p.dc_power, -p.upgrade_kw], hi=p.headroom)
        row([iy[j], iz[j]], [1, -1], hi=0)
    cap = np.r_[np.zeros(m), np.full(n, p.ac_cost), np.full(n, p.dc_cost), np.full(n, p.fixed_cost),
np.full(n, p.upgrade_cost)]
    ann = p.crf * cap
    ann[ix] += p.ac_maintenance + p.network
    ann[idc] += p.dc_maintenance + p.network
    if np.isfinite(budget):
        row(list(range(m, nv)), cap[m:].tolist(), hi=budget)
    if target is not None:
        row(list(range(m)), [1]*m, lo=target)
    # Network-flow upper bound: new supply can add at most q per AC point.
    # The rounded count cut is valid for the integer AC-only target model.
    # It prevents spending branch-and-bound effort proving fractional point counts impossible.
    if target is not None and not p.dc_enabled and not relaxation:
        no_build, _ = allocate(inst, distance=False)
        needed = max(0, int(np.ceil((target-no_build.sum()-1e-7)/p.q)))
        row(ix.tolist(), [1]*n, lo=needed)
    if np.isfinite(budget) and not relaxation and not p.dc_enabled and
min(p.fixed_cost,p.upgrade_cost)>=0:
        row(ix.tolist(), [1]*n, hi=int(np.floor((budget+1e-7)/p.ac_cost)))
    A = coo_matrix((vv, (rr, cc)), shape=(len(lower), nv))
```

```
247      ub = np.r_[np.full(m, np.inf), inst.maximum, inst.maximum if p.dc_enabled else np.zeros(n),
np.ones(2*n)]
248      integer = np.r_[np.zeros(m), np.zeros(4*n) if relaxation else np.ones(4*n)]
249      return A, np.array(lower), np.array(upper), np.zeros(nv), ub, integer, cap, ann
250
251
252  def optimise(inst, budget=np.inf, target=None, name='solve', relaxation=False, fixed_zero=None,
warm_x=None):
253      p, n, m = inst.p, inst.n, len(inst.ii)
254      A, lo, hi, lb, ub, integer, cap, ann = formulation(inst, budget, target, relaxation)
255      if fixed_zero is not None:
256          ub[m + np.array(fixed_zero)] = 0
257          ub[m+n + np.array(fixed_zero)] = 0
258      c = cap / 1000 if target is not None else np.r_[-np.ones(m), np.zeros(4*n)]
259      log = OUT / 'solver_logs' / f'{name}.log'
260      h = make_solver(c, A, lo, hi, lb, ub, integer, p, log)
261      if name == 'base':
262          h.writeModel(str(OUT / 'base_model.lp'))
263      if warm_x is not None:
264          h.setSolution(n, np.arange(m, m+n, dtype=np.int32), np.array(warm_x, dtype=float))
265      if target is None and not relaxation:
266          initial_flow, _ = allocate(inst, distance=False)
267          initial = np.r_[initial_flow, np.zeros(4*n)]
268          h.setSolution(len(c), np.arange(len(c),dtype=np.int32), initial)
269      if target is not None and not relaxation and not p.dc_enabled:
270          # An LP-round-up incumbent is feasible under the integer space and grid bounds.
271          # It supplies a reproducible starting solution without claiming LP decisions are buildable.
272          lp_solver = make_solver(c, A, lo, hi, lb, ub, np.zeros(len(c)), p)
273          relaxed, _ = run(lp_solver, 'target_lp_start')
274          if relaxed is not None:
275              wx = np.minimum(inst.maximum, np.ceil(relaxed[m:m+n]-1e-7)).astype(int)
276              if np.all(wx*p.ac_power <= p.headroom+p.upgrade_kw+1e-7):
277                  wf, _ = allocate(inst, wx, distance=False)
278                  wz, wy, _, _ = investments(inst, wx)
279                  if wf.sum() >= target-1e-5:
280                      start_sol = np.r_[wf, wx, np.zeros(n), wz, wy]
281                      h.setSolution(len(c), np.arange(len(c),dtype=np.int32), start_sol)
282      sol, stage1 = run(h, 'minimum_target_capex' if target is not None else 'maximum_service')
283      stages = [stage1]
284      if sol is None:
285          return {'name': name, 'feasible': False, 'stages': stages, 'budget': budget}
286      first_service = float(sol[:m].sum())
287      epsilon = max(1e-6, 1e-8 * inst.demand.sum())
288      if target is None and not relaxation:
289          h.addRow(first_service - epsilon, np.inf, m, np.arange(m, dtype=np.int32), np.ones(m))
290          if not p.dc_enabled:
291              no_build, _ = allocate(inst, distance=False)
292              needed = max(0, int(np.ceil((first_service-epsilon-no_build.sum()-1e-7)/p.q)))
293              h.addRow(needed, np.inf, n, np.arange(m,m+n,dtype=np.int32), np.ones(n))
294          h.changeColsCost(len(c), np.arange(len(c), dtype=np.int32), ann / 1000)
295          h.setSolution(len(sol), np.arange(len(sol), dtype=np.int32), sol)
296          candidate, stage2 = run(h, 'minimum_annual_cost')
297          stages.append(stage2)
298          if candidate is not None:
299              sol = candidate
300      x, d = sol[m:m+n], sol[m+n:m+2*n]
301      if relaxation:
302          flow = sol[:m]
303          capital, annual = float(cap @ sol), float(ann @ sol)
304          z, y = sol[m+2*n:m+3*n], sol[m+3*n:]
305      else:
306          assert max(np.max(np.abs(x - np.rint(x))), np.max(np.abs(d - np.rint(d)))) < 1e-5
307          x, d = np.rint(x).astype(int), np.rint(d).astype(int)
308          z, y, capitals, annuals = investments(inst, x, d)
309          capital, annual = float(capitals.sum()), float(annuals.sum())
310          flow, _ = allocate(inst, x, d)
311      result = summarise(inst, x, d, flow, budget, name)
312      result.update(feasible=True, capital=capital, annual_cost=annual, stages=stages,
313                    first_service=first_service, service_lock_epsilon=epsilon,
314                    certified_service=stage1['status'] == 'Optimal',
315                    certified_cost=stages[-1]['status'] == 'Optimal', relaxation=relaxation)
316      if relaxation:
317          result['y'], result['z'] = y, z
318          result['site_capital'] = p.ac_cost*x + p.dc_cost*d + p.fixed_cost*z + p.upgrade_cost*y
319          result['site_annual'] = p.crf*result['site_capital'] + (p.ac_maintenance+p.network)*x +
(p.dc_maintenance+p.network)*d
320      return result
321
322
323  def summarise(inst, x, d, flow, budget, name):
324      z, y, capital, annual = investments(inst, x, d)
325      served_i = np.bincount(inst.ii, weights=flow, minlength=inst.n)
326      used_j = np.bincount(inst.jj, weights=flow, minlength=inst.n)
327      total = inst.demand.sum()
328      return {'name': name, 'budget': float(budget), 'demand': float(total), 'served':
float(flow.sum()),
329              'service_fraction': float(flow.sum()/total) if total else 1.,
```

```
330             'capital': float(capital.sum()), 'annual_cost': float(annual.sum()),
331             'ac_points': float(x.sum()), 'dc_points': float(d.sum()), 'sites_selected': int(z.sum()),
332             'upgrades': int(y.sum()), 'unspent': float(budget - capital.sum()),
333             'distance_m': float(inst.distances @ flow / flow.sum()) if flow.sum() else 0,
334             'imputed_investment_share': float(capital[inst.sites.imputed].sum()/capital.sum()) if
capital.sum() else 0,
335             'unallocated_proxy_share': inst.unallocated_proxy_share,
336             'x': x, 'dc': d, 'y': y, 'flow': flow, 'served_i': served_i, 'used_j': used_j}
337
338
339  def validate_result(inst, result, tol=1e-4):
340      if not result.get('feasible', True):
341          return
342      x, d, f = result['x'], result['dc'], result['flow']
343      _, y, cap, _ = investments(inst, x, d)
344      checks = {
345          'nonnegative_flows': max(0., -float(f.min(initial=0))),
346          'demand_balance': max(0., float((result['served_i'] - inst.demand).max(initial=0))),
347          'site_capacity': max(0., float((result['used_j'] - inst.q0 - inst.p.q*x -
inst.p.qdc*d).max(initial=0))),
348          'space': max(0., float((x+d-inst.maximum).max(initial=0))),
349          'power': max(0., float((inst.p.ac_power*x + inst.p.dc_power*d - inst.p.headroom -
inst.p.upgrade_kw*y).max(initial=0))),
350          'budget': max(0., float(cap.sum() - result['budget'])),
351          'integrality': float(max(np.abs(x-np.rint(x)).max(initial=0), np.abs(d-
np.rint(d)).max(initial=0))),
352      }
353      if 'first_service' in result:
354          checks['service_lock'] = max(0., result['first_service'] - result['served'] -
2*result['service_lock_epsilon'])
355      assert all(v <= tol for v in checks.values()), (result['name'], checks)
356      return checks
357
358
359  def save_result(inst, r):
360      dest = OUT / 'runs' / r['name']
361      dest.mkdir(parents=True, exist_ok=True)
362      scalar = {k: v for k, v in r.items() if not isinstance(v, np.ndarray)}
363      scalar['parameters'] = asdict(inst.p)
364      if r.get('feasible', True) and not r.get('relaxation', False):
365          scalar['residual_checks'] = validate_result(inst, r)
366      (dest / 'metrics.json').write_text(json.dumps(scalar, indent=2))
367      if 'x' not in r:
368          return
369      s = inst.sites.copy()
370      s['demand_kwh_day'], s['served_kwh_day'] = inst.demand, r['served_i']
371      s['unmet_kwh_day'] = inst.demand - r['served_i']
372      s['existing_capacity'], s['capacity_used'] = inst.q0, r['used_j']
373      s['ac_additions'], s['dc_additions'], s['upgrade'] = r['x'], r['dc'], r['y']
374      _, _, s['capex'], s['annual_cost'] = investments(inst, r['x'], r['dc'])
375      if r.get('relaxation', False):
376          s['capex'], s['annual_cost'] = r['site_capital'], r['site_annual']
377      s['space_slack'] = inst.maximum - r['x'] - r['dc']
378      s['power_slack_kw'] = inst.p.headroom + inst.p.upgrade_kw*r['y'] - inst.p.ac_power*r['x'] -
inst.p.dc_power*r['dc']
379      s.to_csv(dest / 'site_decisions.csv', index=False)
380      pd.DataFrame({'demand_site': s.car_park_no.iloc[inst.ii].to_numpy(), 'supply_site':
s.car_park_no.iloc[inst.jj].to_numpy(),
381                    'distance_m': inst.distances, 'kwh_day': r['flow']}).query('kwh_day >
1e-8').to_csv(dest / 'demand_flows.csv', index=False)
382      s.groupby('area_2019')[['demand_kwh_day', 'served_kwh_day', 'unmet_kwh_day', 'ac_additions',
'dc_additions', 'capex', 'annual_cost']].sum().to_csv(dest / 'area_results.csv')
```

Appendix C. Python — baselines.py

```
"""Feasible equal, largest-remainder proportional, and exact-marginal greedy rules."""
import heapq
import numpy as np
from scipy.sparse import coo_matrix
from model import make_solver, run, investments, allocate, summarise


def step_cost(inst, x, j):
    p = inst.p
    if x[j] + 1 > inst.maximum[j] or (x[j]+1)*p.ac_power > p.headroom + p.upgrade_kw + 1e-7:
        return np.inf
    before = x[j]*p.ac_power > p.headroom + 1e-7
    after = (x[j]+1)*p.ac_power > p.headroom + 1e-7
    return p.ac_cost + (p.fixed_cost if x[j] == 0 else 0) + (p.upgrade_cost if after and not before
else 0)


def simple_rule(inst, budget, kind):
    x = np.zeros(inst.n, dtype=int)
    remaining = budget
    if kind == 'equal':
        while True:
            changed = False
            for j in range(inst.n):
                cost = step_cost(inst, x, j)
                if cost <= remaining + 1e-7:
                    x[j] += 1; remaining -= cost; changed = True
            if not changed:
                break
    elif kind == 'proportional':
        # Hamilton largest remainder for the nominal count; repair under actual step costs.
        w = inst.sites.lots.to_numpy() / inst.sites.lots.sum()
        nominal = int(budget // inst.p.ac_cost)
        ideal = nominal * w
        target = np.floor(ideal).astype(int)
        for j in np.lexsort((np.arange(inst.n), -(ideal-target)))[:nominal-target.sum()]:
            target[j] += 1
        for j in range(inst.n):
            for _ in range(target[j]):
                cost = step_cost(inst, x, j)
                if cost > remaining + 1e-7:
                    break
                x[j] += 1; remaining -= cost
        while True:
            candidates = [j for j in range(inst.n) if step_cost(inst, x, j) <= remaining + 1e-7]
            if not candidates:
                break
            j = min(candidates, key=lambda j: (-(ideal[j]-x[j]), j))
            remaining -= step_cost(inst, x, j); x[j] += 1
    else:
        raise ValueError(kind)
    flow, _ = allocate(inst, x)
    r = summarise(inst, x, np.zeros(inst.n), flow, budget, kind)
    r['algorithm'] = 'ID-ordered equal rounds' if kind == 'equal' else 'Hamilton largest remainder +
feasible deficit repair'
    return r


def greedy_rule(inst, budget):
    """Lazy exact max-flow marginal per current step dollar, including upgrade thresholds.

    Max-flow served demand is submodular in added source capacities. Cached marginals
    are upper bounds as other site capacities grow. The selected site's step cost is
    recomputed after every addition, so an upgrade threshold cannot reuse an old price.
    Each tested marginal re-solves all allocations using a warm-started LP.
    """
    n, m, p = inst.n, len(inst.ii), inst.p
    A = coo_matrix((np.ones(2*m), (np.r_[inst.ii, n+inst.jj], np.tile(np.arange(m), 2))), shape=(2*n,
m))
    h = make_solver(-np.ones(m), A, np.zeros(2*n), np.r_[inst.demand, inst.q0], np.zeros(m),
np.full(m, np.inf), p=p)
    _, info = run(h, 'greedy_initial')
    service = -h.getObjectiveValue()
    x = np.zeros(n, dtype=int)
    remaining, generation, evaluations = budget, 0, 0
    heap = []
    for j in range(n):
        cost = step_cost(inst, x, j)
        if cost <= remaining:
            heapq.heappush(heap, (-p.q/cost, j, -1))
```

```
    trace = []
    while heap:
        negative_bound, j, age = heapq.heappop(heap)
        cost = step_cost(inst, x, j)
        if cost > remaining + 1e-7:
            continue
        oldcap = inst.q0[j] + p.q*x[j]
        h.changeRowBounds(n+j, 0, oldcap+p.q)
        _, info = run(h, 'greedy_marginal')
        assert info['status'] == 'Optimal'
        trial = -h.getObjectiveValue()
        gain = max(0., trial-service)
        ratio = gain/cost
        evaluations += 1
        if ratio + 1e-10 >= (-heap[0][0] if heap else 0):
            if gain <= 1e-6:
                h.changeRowBounds(n+j, 0, oldcap)
                break
            x[j] += 1; remaining -= cost; generation += 1; service = trial
            trace.append({'step': generation, 'site_id': inst.sites.car_park_no.iloc[j], 'step_capex':
cost, 'marginal_kwh_day': gain, 'cumulative_capex': budget-remaining})
            newcost = step_cost(inst, x, j)
            if newcost <= remaining + 1e-7:
                heapq.heappush(heap, (-p.q/newcost, j, -1))
        else:
            h.changeRowBounds(n+j, 0, oldcap)
            _, back = run(h, 'greedy_restore')
            heapq.heappush(heap, (-ratio, j, generation))
    flow, _ = allocate(inst, x)
    assert abs(flow.sum()-service) < .02
    r = summarise(inst, x, np.zeros(n), flow, budget, 'greedy')
    r.update(marginal_lp_evaluations=evaluations, algorithm='Exact reassigned-flow marginal/step-cost
greedy; lazy upper bounds')
    return r, trace
```

Appendix C. Python — run_analysis.py

```
001  """Reproduce the frozen national analysis, comparisons, and fixed-plan stresses."""
002  import argparse
003  from dataclasses import asdict, replace
004  from pathlib import Path
005  import hashlib
006  import json
007  import platform
008  import time
009  import multiprocessing
010  from concurrent.futures import ProcessPoolExecutor, as_completed
011  import numpy as np
012  import pandas as pd
013  import highspy
014  from model import Parameters, build_instance, optimise, allocate, summarise, save_result, investments
015  from baselines import simple_rule, greedy_rule
016  from prepare_data import INPUTS, OUT
017
018
019  def cache_key(inst, budget, target, relaxation, fixed_zero, implementation=None):
020      source = implementation or
''.join(hashlib.sha256(Path(__file__).with_name(p).read_bytes()).hexdigest() for p in
['model.py','prepare_data.py','baselines.py'])
021      source += (INPUTS/'source_hashes.json').read_text()
022      return hashlib.sha256((source + json.dumps([asdict(inst.p), budget, target, relaxation,
fixed_zero], sort_keys=True)).encode()).hexdigest()
023
024
025  def solve(inst, name, budget=np.inf, target=None, relaxation=False, fixed_zero=None):
026      key = cache_key(inst, budget, target, relaxation, fixed_zero)
027      folder = OUT/'runs'/name
028      if (folder/'metrics.json').exists() and (folder/'arrays.npz').exists():
029          old = json.loads((folder/'metrics.json').read_text())
030          compatible = json.loads(Path(__file__).with_name('cache_compatibility.json').read_text())
['compatible_implementation_fingerprints']
031          keys = [key] + [cache_key(inst,budget,target,relaxation,fixed_zero,implementation=v) for v in
compatible]
032          if old.get('cache_key') in keys:
033              old.update(dict(np.load(folder/'arrays.npz')))
034              print('CACHED',name,flush=True)
035              return old
036      print('SOLVE', name, 'budget', budget, 'target', target, flush=True)
037      r = optimise(inst, budget, target, name, relaxation, fixed_zero)
038      r['cache_key'] = key
039      save_result(inst, r)
040      if 'x' in r:
041          np.savez_compressed(folder/'arrays.npz', **{k:v for k,v in r.items() if
isinstance(v,np.ndarray)})
042          print('RESULT',name,round(r['service_fraction']*100,3),'%',round(r['capital']),
[(s['status'],round(s['seconds'],2),s['gap']) for s in r['stages']],flush=True)
043      else:
044          print('INFEASIBLE/NO INCUMBENT',name,r['stages'],flush=True)
045      return r
046
047
048  def flat(r):
049      out = {k:v for k,v in r.items() if np.isscalar(v) and k not in ['cache_key']}
050      if 'stages' in r:
051          for j, st in enumerate(r['stages'],1):
052              out.update({f'pass{j}_{k}': v for k,v in st.items() if k in
['status','gap','bound','seconds']})
053      return out
054
055
056  def target_solve(inst, fraction):
057      """Try a constructive minimum-cost certificate before a harder target MILP.
058
059      Every new AC point can add at most q service. Thus at least
060      ceil((target-S0)/q) points are needed, even with perfect placement.
061      Nonnegative opening/upgrade costs imply capex >= that count * point cost.
062      A feasible plan attaining this cost proves minimum capex independently
063      of branch-and-bound. If unattainable, use the general target MILP.
064      """
065      f0,_=allocate(inst,distance=False)
066      target=fraction*inst.demand.sum()
067      count=max(0,int(np.ceil((target-f0.sum()-1e-7)/inst.p.q)))
068      bound=count*inst.p.ac_cost
069      trial=solve(inst,f'target_{int(fraction*100)}_capacity_bound',budget=bound)
070      if trial.get('feasible') and trial['served']>=target-1e-5:
071          r=trial.copy();r['name']=f'target_{int(fraction*100)}'
072          r['target_fraction']=fraction;r['target_kwh_day']=target
```

```
073          r['minimum_capex_proof']='Feasible plan attains ceil((target-S0)/q)*c lower bound; no
upgrade/opening premium'
074          r['minimum_capex_bound']=bound;r['certified_target_capex']=True
075          r['stages']=trial['stages']+
[{'stage':'target_capex_lower_bound_certificate','status':'Optimal','objective':bound,'bound':bound,'gap':0
.,'nodes':0,'seconds':0.}]
076          save_result(inst,r)
077          return r
078      return solve(inst,f'target_{int(fraction*100)}',target=target)
079
080
081  def fixed_plan(inst, base, budget):
082      x, d, y = base['x'], base['dc'], base['y']
083      power = inst.p.ac_power*x + inst.p.dc_power*d
084      available = inst.p.headroom + inst.p.upgrade_kw*y
085      scale = np.minimum(1., np.divide(available, power, out=np.ones(inst.n), where=power>0))
086      scale = np.minimum(scale, np.divide(inst.maximum, x+d, out=np.ones(inst.n), where=x+d>0))
087      q = inst.q0 + scale*(inst.p.q*x + inst.p.qdc*d)
088      f, _ = allocate(inst, q_override=q)
089      cap = inst.p.ac_cost*x + inst.p.dc_cost*d + inst.p.fixed_cost*(x+d>0) + inst.p.upgrade_cost*y
090      return {'fixed_served': float(f.sum()), 'fixed_fraction': float(f.sum()/inst.demand.sum()),
091              'fixed_repriced_capital': float(cap.sum()), 'fixed_over_budget':
bool(cap.sum()>budget+1e-5),
092              'fixed_derated_sites': int((scale<1-1e-7).sum()),
093              'fixed_construction_feasible': bool(np.all(scale>=1-1e-7) and cap.sum()<=budget+1e-5)}
094
095
096  def sensitivity_case(name,delta,p,base,budget):
097      pp=replace(p,**delta,time_limit=25,mip_gap=.001)
098      si=build_instance(pp)
099      r=solve(si,name,budget=budget)
100      params={'scenario':name,**asdict(pp)}
101      if not r.get('feasible'):
102          return flat(r),None,params
103      fixed=fixed_plan(si,base,budget)
104      zero,_=allocate(si,distance=False)
105
row={**flat(r),**fixed,'no_build_served':float(zero.sum()),'reoptimised_minus_fixed_kwh':r['served']-
fixed['fixed_served']}
106      row['feasible_fixed_plan_regret_kwh']=max(0.,row['reoptimised_minus_fixed_kwh']) if
fixed['fixed_construction_feasible'] else np.nan
107      return row,(r['x']+r['dc'])>0,params
108
109
110  def finish_outputs(inst,base,budget):
111      """Rank only certified scenario allocations; retain all-case frequency separately."""
112      sensitivity=pd.read_csv(OUT/'sensitivity.csv')
113      all_selected=[];certified=[]
114      for row in sensitivity.itertuples():
115
s=pd.read_csv(OUT/'runs'/row.name/'site_decisions.csv').set_index('car_park_no').reindex(inst.sites.car_par
k_no)
116          chosen=(s.ac_additions.to_numpy()+s.dc_additions.to_numpy())>0
117          all_selected.append(chosen)
118          if row.certified_service and row.certified_cost:certified.append(chosen)
119      assert certified,'No certified scenario set available for shortlist frequency'
120      rob=inst.sites[['car_park_no','address','area_2019','imputed','night_parking']].copy()
121      rob['selection_fraction']=np.mean(certified,axis=0)
122      rob['scenario_count']=len(certified)
123      rob['all_feasible_selection_fraction']=np.mean(all_selected,axis=0)
124      rob['all_feasible_scenario_count']=len(all_selected)
125      rob['base_additions']=base['x'];rob['base_upgrade']=base['y']
126      rob['base_capex']=investments(inst,base['x'],base['dc'])[2]
127
rob.sort_values(['selection_fraction','base_additions','car_park_no'],ascending=[False,False,True]).to_csv(
OUT/'site_robustness.csv',index=False)
128
shortlist=rob[rob.base_additions.gt(0)&~rob.imputed&rob.night_parking.eq('YES')].sort_values(['selection_fr
action','base_additions'],ascending=False).head(5)
129      replacement=[]
130      for j,row in shortlist.iterrows():
131          rr=solve(inst,f'remove_{row.car_park_no}',budget=budget,fixed_zero=[int(j)])
132
replacement.append({'site_id':row.car_park_no,'base_additions':row.base_additions,'selection_fraction':row.
selection_fraction,
133                              'certified_scenario_count':len(certified),
134                              'replacement_service_loss_kwh_day':base['served']-rr.get('served',np.nan),
135                              'status':rr['stages'][0]['status'],'service_gap':rr['stages'][0]['gap']})
136      pd.DataFrame(replacement).to_csv(OUT/'shortlist_replacement.csv',index=False)
137      alternate=solve(build_instance(replace(inst.p,seed=29)), 'alternate_seed',budget=budget)
138      repeat=solve(inst,'repeat_base',budget=budget)
139      again=solve(inst,'repeat_current_2',budget=budget)
140      overlap=lambda a,b: float(np.sum((a['x']>0)&(b['x']>0))/max(1,np.sum((a['x']>0)|(b['x']>0))))
141      (OUT/'reproducibility.json').write_text(json.dumps({'repeat_service_difference':again['served']-
repeat['served'],
142          'repeat_identical_additions':bool(np.array_equal(again['x'],repeat['x'])),
143          'alternate_seed_service_difference':alternate['served']-repeat['served'],
```

```
144
'alternate_seed_site_jaccard':overlap(alternate,repeat),'repeat_site_jaccard':overlap(again,repeat),
145         'frozen_base_vs_current_implementation_jaccard':overlap(base,repeat)},indent=2))
146
147
148  def main(mode='all',workers=3):
149      OUT.mkdir(parents=True,exist_ok=True)
150      p = Parameters()
151      stock = pd.read_csv(INPUTS/'ev_stock_scenarios.csv').set_index('scenario').stock
152      p = replace(p, stock=float(stock['2027_base']))
153      inst = build_instance(p)
154      pd.DataFrame({'demand_site':inst.sites.car_park_no.iloc[inst.ii].to_numpy(),
155                    'supply_site':inst.sites.car_park_no.iloc[inst.jj].to_numpy(),
'distance_m':inst.distances}).to_csv(INPUTS/'arcs.csv',index=False)
156      pd.DataFrame({'site_id':inst.sites.car_park_no,'weight':inst.demand/
inst.demand.sum(),'hdb_evs':p.stock*p.hdb_share*inst.demand/inst.demand.sum(),
'demand_kwh_day':inst.demand}).to_csv(INPUTS/'demand.csv',index=False)
157      f0, _ = allocate(inst)
158      baseline = summarise(inst,np.zeros(inst.n),np.zeros(inst.n),f0,0,'no_build')
159      save_result(inst,baseline)
160      max_x = np.minimum(inst.maximum,np.floor((p.headroom+p.upgrade_kw)/p.ac_power+1e-8))
161      maxflow,_=allocate(inst,max_x)
162      maximum=float(maxflow.sum())
163      target_rows=[]
164      for target in [.8,.9,.95]:
165          r=target_solve(inst,target)
166          target_rows.append(flat(r))
167          if target==.9:
168              anchor=r
169      pd.DataFrame(target_rows).to_csv(OUT/'service_targets.csv',index=False)
170      if anchor.get('feasible') and anchor['capital']>0:
171          budget=anchor['capital']; anchor_reason='Minimum-capex 90% service target (solver status/gap
retained)'
172      else:
173          near=solve(inst,'target_near_maximum',target=max(baseline['served'],maximum*.995))
174          budget=near['capital']; anchor_reason='90% is already met or infeasible; 99.5% of physical
maximum instead'
175      base=solve(inst,'base',budget=budget)
176      if not base.get('feasible'):
177          raise RuntimeError('No base investment incumbent')
178      base['no_build_served']=baseline['served']
179      base['incremental_served']=base['served']-baseline['served']
180      info={'anchor_reason':anchor_reason,'budget':budget,'maximum_physical_service':maximum,
181            'maximum_physical_fraction':maximum/inst.demand.sum(), 'no_build_served':baseline['served'],
182            'no_build_fraction':baseline['service_fraction'],'base_served':base['served'],
183            'incremental_served':base['incremental_served'], 'q_new_ac':p.q,'arcs':len(inst.ii)}
184      (OUT/'analysis_summary.json').write_text(json.dumps(info,indent=2))
185      if mode in ['all','core']:
186          frontier=[flat(baseline)]
187          for multiple in [.25,.5,.75,1.,1.25]:
188              r=base if multiple==1 else solve(inst,f'frontier_{multiple:g}',budget=budget*multiple)
189              row=flat(r);row['budget_multiple']=multiple;frontier.append(row)
190          df=pd.DataFrame(frontier).sort_values('budget')
191          df['incremental_kwh_day']=df.served-baseline['served']
192
df['annual_infrastructure_cost_per_extra_kwh']=df.annual_cost/(365*df.incremental_kwh_day.replace(0,np.nan)
)
193          df['marginal_capex_per_extra_annual_kwh']=df.capital.diff()/(365*df.served.diff())
194          df.to_csv(OUT/'budget_frontier.csv',index=False)
195          assert np.all(np.diff(df.served)>-max(1e-3,inst.demand.sum()*p.mip_gap)), 'Nonmonotone
frontier beyond solver tolerance'
196          comparisons=[flat(baseline),flat(base)]
197          for kind in ['equal','proportional','greedy']:
198              if kind=='greedy':
199                  print('GREEDY exact reassignment starting',flush=True)
200                  r,trace=greedy_rule(inst,budget)
201                  pd.DataFrame(trace).to_csv(OUT/'greedy_steps.csv',index=False)
202              else:
203                  r=simple_rule(inst,budget,kind)
204              save_result(inst,r)
205              comparisons.append(flat(r))
206              print('COMPARISON',kind,r['capital'],r['service_fraction'],flush=True)
207              matched=solve(inst,f'matched_{kind}',budget=r['capital'])
208              comparisons.append(flat(matched))
209          pd.DataFrame(comparisons).to_csv(OUT/'baseline_comparison.csv',index=False)
210          relaxation=solve(inst,'lp_relaxation',budget=budget,relaxation=True)
211          (OUT/'lp_bound.json').write_text(json.dumps({'lp_service_upper_bound':relaxation['served'],
212              'integer_service':base['served'], 'absolute_bound_gap_kwh_day':relaxation['served']-
base['served']},indent=2))
213      if mode in ['all','sensitivity']:
214          changes={
215              'stock_current_2026':{'stock':stock['current_2026']},
216              'stock_2027_low':{'stock':stock['2027_low']},'stock_2027_high':
{'stock':stock['2027_high']},
217              'outlook_2030_low':{'stock':stock['2030_low']},'outlook_2030_base':
{'stock':stock['2030_base']},'outlook_2030_high':{'stock':stock['2030_high']},
218              'hdb_share_30':{'hdb_share':.3},'hdb_share_55':{'hdb_share':.55},
```

```
219              'dependence_60':{'dependence':.6},'dependence_100':{'dependence':1.},
220              'mileage_12000':{'annual_km':12000},'mileage_20000':{'annual_km':20000},
221              'intensity_014':{'intensity':.14},'intensity_020':{'intensity':.2},
222              'efficiency_085':{'efficiency':.85},'efficiency_095':{'efficiency':.95},
223              'unknown_no_build':{'unknown_no_build':True},'imputed_half':
{'impute_factor':.5},'imputed_150':{'impute_factor':1.5},
224              'housing_2025':{'weights':'housing_2025'},'population_2025':{'weights':'population_2025'},
225              'housing_2026':{'weights':'housing_2026'},'population_2026':{'weights':'population_2026'},
226              'broad_supply':{'matching':'broad'},'sum_port_power':{'power_bound':'upper'},
227              'hours_4':{'hours':4},'hours_8':{'hours':8},
228              'availability_90':{'availability':.9},'availability_99':{'availability':.99},
229              'mean_power_70':{'power_factor':.7},'mean_power_100':{'power_factor':1.},
230              'radius_250':{'radius':250},'radius_800':{'radius':800},'self_site_only':{'radius':0},
231              'path_inflation_140':{'path_inflation':1.4},
232              'capex_4500':{'ac_cost':4500},'capex_10000':{'ac_cost':10000},
233              'headroom_148':{'headroom':14.8},'headroom_888':{'headroom':88.8},
234              'upgrade_free':{'upgrade_cost':0},'upgrade_250k':{'upgrade_cost':250000},
235              'lifetime_8':{'lifetime':8},'lifetime_12':{'lifetime':12},
236              'discount_03':{'discount':.03},'discount_08':{'discount':.08},
237              'lot_share_05':{'lot_share':.05},'lot_share_15':{'lot_share':.15},
238              'deduplicated_bays':{'deduplicate_bays':True},
239              'ac_dc_extension':{'dc_enabled':True},
240              'joint_low':{'stock':stock['2027_low'],'ac_cost':4500,'hours':8},
241              'joint_high':
{'stock':stock['2027_high'],'hdb_share':.55,'ac_cost':10000,'headroom':14.8,'upgrade_cost':250000},
242              'joint_access':{'radius':0,'unknown_no_build':True},
243          }
244          rows=[];selected=[]
245          params=[]
246          with ProcessPoolExecutor(max_workers=workers,mp_context=multiprocessing.get_context('spawn'))
as pool:
247              futures=[pool.submit(sensitivity_case,name,delta,p,base,budget) for name,delta in
changes.items()]
248              for future in as_completed(futures):
249                  row,selection,parameter=future.result()
250                  rows.append(row);params.append(parameter)
251                  if selection is not None:selected.append(selection)
252                  pd.DataFrame(rows).to_csv(OUT/'sensitivity.csv',index=False)
253          order={name:k for k,name in enumerate(changes)}
254          rows.sort(key=lambda row:order[row['name']]);params.sort(key=lambda
row:order[row['scenario']])
255          pd.DataFrame(rows).to_csv(OUT/'sensitivity.csv',index=False)
256          pd.DataFrame(params).to_csv(OUT/'scenario_parameters.csv',index=False)
257      if mode in ['all','sensitivity','finalize']:
258          finish_outputs(inst,base,budget)
259      manifest={'executed_at_utc':time.strftime('%Y-%m-%dT%H:%M:
%SZ',time.gmtime()),'python':platform.python_version(),
260                'highs_version':highspy.Highs().version(),'base_parameters':asdict(p),'mode':mode,
261                'solver_threads':1,'sensitivity_worker_processes':workers,'stock_paths':'assumed, not
fitted','monetary_basis':'2026 scenario SGD excluding GST; constant real costs',
262                'code_hashes':{f.name:hashlib.sha256(f.read_bytes()).hexdigest() for f in
Path(__file__).parent.glob('*.py')}}
263      (OUT/'run_manifest.json').write_text(json.dumps(manifest,indent=2))
264      print('COMPLETE',mode,flush=True)
265
266
267  if __name__=='__main__':
268      parser=argparse.ArgumentParser();parser.add_argument('--
mode',choices=['all','core','sensitivity','finalize'],default='all')
269      parser.add_argument('--workers',type=int,default=3)
270      args=parser.parse_args();main(args.mode,args.workers)
```

Appendix C. Python — test_models.py

```
"""Small independent expected-answer checks, plus source-data reconciliation."""
import unittest
from dataclasses import replace
import numpy as np
import pandas as pd
from model import Parameters, Instance, optimise, allocate, build_instance, validate_result
from prepare_data import INPUTS, address_parts
from baselines import greedy_rule


def tiny(demand=(5., 5.), shared=True, max_points=3):
    s = pd.DataFrame({'car_park_no': ['A', 'B'], 'imputed': [False, False], 'lots': [10, 10]})
    p = Parameters(ac_power=1, hours=4, availability=1, power_factor=1,
                   ac_cost=3, headroom=1, upgrade_kw=2, upgrade_cost=5,
                   fixed_cost=0, ac_maintenance=0, network=0, time_limit=10, mip_gap=0)
    ii, jj = np.array([(0, 0), (0, 1), (1, 0), (1, 1)] if shared else [(0, 0), (1, 1)]).T
    return Instance(s, p, np.array(demand), np.array([4., 0.]), np.array([1., 0.]),
                    np.array([max_points, 0]), ii, jj, np.where(ii==jj, 0., 100.))


class ModelTests(unittest.TestCase):
    def test_hand_solved_upgrade_integrality(self):
        i = tiny()
        # Initial 4; one point costs 3 and serves 8; two cost 11 and serve all 10.
        for budget, served, count in [(0, 4, 0), (3, 8, 1), (10, 8, 1), (11, 10, 2)]:
            r = optimise(i, budget=budget, name=f'test_tiny_{budget}')
            self.assertAlmostEqual(r['served'], served, places=4)
            self.assertEqual(r['x'].sum(), count)
            validate_result(i, r)

    def test_disconnected_node_remains_unmet(self):
        i = tiny(shared=False)
        i.ii, i.jj, i.distances = np.array([0]), np.array([0]), np.array([0.])
        r = optimise(i, budget=100, name='test_disconnected')
        self.assertAlmostEqual(r['served'], 5, places=4)
        self.assertEqual(r['served_i'][1], 0)

    def test_zero_demand_buys_nothing(self):
        r = optimise(tiny((0., 0.)), budget=100, name='test_zero_demand')
        self.assertEqual(r['capital'], 0)

    def test_infeasible_target(self):
        r = optimise(tiny(shared=False), target=9., name='test_infeasible')
        self.assertFalse(r['feasible'])
        self.assertEqual(r['stages'][0]['status'], 'Infeasible')

    def test_monotonic_cost_capacity_and_lp_bound(self):
        i = tiny()
        r = optimise(i, budget=6, name='test_integer')
        lp = optimise(i, budget=6, name='test_relaxation', relaxation=True)
        self.assertGreaterEqual(lp['served']+1e-5, r['served'])
        cheaper = replace(i, p=replace(i.p, ac_cost=1, upgrade_cost=0))
        self.assertGreaterEqual(optimise(cheaper, budget=6, name='test_cheap')['served']+1e-5,
r['served'])
        capacity = replace(i, q0=i.q0+3)
        self.assertGreaterEqual(allocate(capacity)[0].sum(), allocate(i)[0].sum())

    def test_greedy_step_upgrade(self):
        r, trace = greedy_rule(tiny(), 11)
        self.assertAlmostEqual(r['served'], 10, places=4)
        self.assertEqual([t['step_capex'] for t in trace], [3, 8])

    def test_fixed_site_cost_and_dc_choice(self):
        i=tiny()
        activated=replace(i,p=replace(i.p,fixed_cost=2))
        r=optimise(activated,budget=3,name='test_activation_cost')
        self.assertEqual(r['x'].sum(),0)
        dc=replace(i,p=replace(i.p,dc_enabled=True,dc_power=2,dc_hours=4,
                               dc_power_factor=1,dc_cost=4,dc_maintenance=0))
        r=optimise(dc,budget=9,name='test_dc_choice')
        self.assertEqual(r['dc'].sum(),1)
        self.assertEqual(r['x'].sum(),0)
        self.assertAlmostEqual(r['served'],10,places=4)
        self.assertEqual(r['capital'],9)
        validate_result(dc,r)

    def test_source_reconciliation(self):
        i = build_instance()
        s = i.sites
        c = pd.read_csv(INPUTS / 'hdb_charger_crosswalk.csv')
```

```
080          self.assertEqual(len(s), 2273)
081          self.assertEqual(int(s.imputed.sum()), 278)
082          self.assertEqual(s.total_lots.sum(), 706663)
083          self.assertTrue(c.evCpId.is_unique)
084          self.assertEqual(len(c), 11489)
085          a = pd.read_csv(INPUTS / 'area_demand_weights.csv')
086          for y in (2025, 2026):
087              stock = a[(a.year==y)&(a.metric=='dwellings')].value.sum()
088              self.assertTrue(1e6 < stock < 1.5e6, stock)
089          self.assertTrue(np.isfinite(i.demand).all())
090          expected = i.p.stock * i.p.hdb_share * i.p.dependence *
i.p.annual_km/365*i.p.intensity/i.p.efficiency
091          self.assertAlmostEqual(i.demand.sum(), expected)
092          self.assertEqual(set(zip(i.ii[i.ii==i.jj],i.jj[i.ii==i.jj])), {(j,j) for j in range(i.n)})
093
094      def test_address_parser(self):
095          b, s, r = address_parts('BLK 720-727,730-731 CLEMENTI WEST STREET 2')
096          self.assertEqual(s, 'CLEMENTI WEST STREET 2')
097          self.assertEqual(b, {'720','727','730','731'})
098          self.assertIn((720,727),r)
099
100
101  if __name__ == '__main__':
102      unittest.main(verbosity=2)
```